

Phase 2: Software Engineering & Distribution

Goal: Mastering the tools and workflows used by professional developers in global tech firms.

Module 1: Environment Mastery & Path Architecture

- **Virtual Environments (venv, conda):**
 - *Explanation:* Creating isolated project "bubbles" so that different library versions don't conflict.
- **The Global PATH & Interpreter:**
 - *Explanation:* Understanding how the Operating System finds Python and how to fix "Command Not Found" errors.
- **Python Path (sys.path):**
 - *Explanation:* Learning how Python locates your custom scripts and modules across different folders.

Module 2: Advanced Dependency Management

- **Pip & Requirements Files:**
 - *Explanation:* Generating requirements.txt to ensure anyone can replicate the project setup.
- **Modern Managers (Poetry, Pipenv):**
 - *Explanation:* Using advanced tools for "lock files" which guarantee that every developer uses the *exact* same version of a library.
- **Mirroring & Caching:**
 - *Explanation:* How to manage library installations in environments with slow or restricted internet (common in some KP labs).

Module 3: Industrial Coding Standards (PEP 8)

- **Style Guides (PEP 8 / PEP 257):**
 - *Explanation:* Learning the international rules for naming variables and formatting code.

- **Automated Linters (Flake8, Pylint):**
 - *Explanation:* Tools that scan code and point out "ugly" or "bad" logic before you run it.
- **Auto-Formatters (Black, isort):**
 - *Explanation:* One-click tools that automatically rewrite your code to look like it was written by a senior engineer.

Module 4: Professional Version Control (Git CLI)

- **The Git Lifecycle:**
 - *Explanation:* Mastering add, commit, push, and pull from the terminal, not just the GitHub website.
- **Branching & Merging:**
 - *Explanation:* Working on a "New Feature" branch without breaking the main "Working" code.
- **Conflict Resolution:**
 - *Explanation:* What to do when two developers change the same line of code at the same time.

Module 5: OS Interaction & File Engineering

- **The os and shutil Modules:**
 - *Explanation:* Moving, renaming, copying, and deleting thousands of files with 5 lines of code.
- **Pathlib (Modern Pathing):**
 - *Explanation:* Handling file paths correctly so a script written on Windows also works on Linux/Mac.
- **File Permissions & Metadata:**
 - *Explanation:* Reading "hidden" file data like creation time, size, and owner permissions.

Module 6: Automation & Process Management

- The subprocess Module:
 - *Explanation:* Using Python to "remote control" other apps like FFmpeg (for video) or Git itself.
- Running Shell Commands:
 - *Explanation:* Capturing the output of a terminal command and using it inside your Python script.
- System Exit Codes:
 - *Explanation:* Teaching the program to tell the computer *why* it stopped (Success vs. Error codes).

Module 7: Advanced Built-in Tooling

- The collections Module:
 - *Explanation:* Using namedtuple, deque, and Counter for much faster data processing than standard lists.
- itertools for Efficiency:
 - *Explanation:* Creating complex loops (like every possible combination of a list) without using massive amounts of RAM.
- Regular Expressions (re):
 - *Explanation:* Searching for patterns in text (e.g., finding every email address or phone number in a 5,000-page document).

Module 8: Command Line Interface (CLI) Building

- argparse Basics:
 - *Explanation:* Allowing users to type commands like `python my_tool.py --input data.csv`.
- Modern CLI Frameworks (Click, Typer):
 - *Explanation:* Building colorful, professional terminal apps with auto-generated "Help" menus.
- Standard Input/Output (Stdin/Stdout):

- *Explanation:* Allowing your Python script to "pipe" data into other programs.

Module 9: Logging & Diagnostics

- The logging Library:
 - *Explanation:* Replacing print() with a system that saves errors to a .log file for later review.
- Log Levels:
 - *Explanation:* Categorizing messages as DEBUG, INFO, WARNING, ERROR, or CRITICAL.
- Rotating Logs:
 - *Explanation:* Ensuring log files don't grow so big that they fill up the computer's hard drive.

Module 10: Network & API Integration

- HTTP Basics with requests:
 - *Explanation:* How to Send/Receive data from the web (GET, POST, PUT, DELETE).
- JSON Manipulation:
 - *Explanation:* Converting Python dictionaries into JSON strings and back—the standard for all web data.
- API Authentication:
 - *Explanation:* Using "API Keys" and "Tokens" to securely access private data from Google, Meta, or OpenAI.
- The requests library: GET, POST, PUT, DELETE.
- Handling JSON payloads and headers.

Module 11: Introduction to Web Services (FastAPI Foundations) [NEW]

- What is an API? (Client vs. Server architecture).
- Building your first GET and POST endpoints with FastAPI.
- Uvicorn and local hosting.

Module 12: Concurrency (Threading & Multiprocessing)

- **Multi-threading:**
 - *Explanation:* Running multiple tasks that "wait" for the internet at the same time (e.g., downloading 50 files).
- **Multiprocessing:**
 - *Explanation:* Using all 4 or 8 cores of a CPU for heavy mathematical calculations.
- **The GIL (Global Interpreter Lock):**
 - *Explanation:* Understanding the "traffic cop" inside Python that limits how fast code can run.

Module 13: Asynchronous Programming (asyncio)

- **The Event Loop:**
 - *Explanation:* Writing non-blocking code that can handle thousands of connections at once.
- **async and await:**
 - *Explanation:* The modern syntax used in every high-performance AI and Web framework today.
- **Non-blocking I/O:**
 - *Explanation:* Ensuring your app doesn't "freeze" while waiting for a slow database response.

Module 14: Automated Testing & Test-Driven Development

- **Unit Testing (unittest):**
 - *Explanation:* Writing "mini-scripts" that automatically check if your main code is still working.
- **Professional Testing with Pytest:**
 - *Explanation:* The industry-standard tool for running thousands of tests in seconds.
- **Mocking & Patching:**

- *Explanation:* Simulating a database or an internet connection so you can test your code even when you are offline.

Module 15: Packaging & Global Distribution (The "Cap Stone")

- **Library Structuring:**
 - *Explanation:* Organizing code into a structure that can be shared via import.
- **pyproject.toml Configuration:**
 - *Explanation:* Writing the "ID card" for your library (Name, Version, Author).
- **PyPI Publishing with Twine:**
 - *Explanation:* Uploading the project so anyone in the world can run `pip install [your-name-here]`.

Module 16:

- **Containerization Basics (Docker)**
 - *Docker & docker-compose: Why every AI service and micro-service in 2026 runs in containers.*
 - *Building and running your first Python container.*
 - *Portability across Windows, Linux, and Mac*

Phase 2: Professional System Engineering Tasks

Task 1: The Multi-threaded Image Dataset Scraper

- **The Task:** Create a tool that downloads 50 high-resolution images from a list of URLs simultaneously.
- **Deep Details:**

- **Concurrency:** Use the threading module or `concurrent.futures.ThreadPoolExecutor` to download images in parallel rather than one by one.
- **IO Management:** Implement requests with a custom "User-Agent" to avoid being blocked by servers.
- **Phase 3 Bridge:** This is the standard method for collecting raw data for Image Segmentation and Computer Vision projects.

Task 2: The "PyPI-Style" Package Architecture

- **The Task:** Take a complex logic (like a custom math library or a string cleaner) and turn it into an installable Python package.
- **Deep Details:**
 - **Structure:** Organize the folder with a `src/` directory, an `__init__.py` for namespace control, and a `README.md`.
 - **Build System:** Write a `pyproject.toml` or `setup.py` file.
 - **Installation:** The student must successfully run `pip install -e .` (editable mode) and import their library from a completely different directory on the system.

Task 3: Automated API Wrapper with Logging

- **The Task:** Build a Class-based wrapper that talks to a live Weather or Finance API.
- **Deep Details:**
 - **Diagnostics:** Remove all `print()` statements. Use the logging module to record API status codes to a `system.log` file.
 - **Security:** Use `python-dotenv` to load API keys from a `.env` file (never hardcode keys).
 - **Stability:** Implement a "Retry" logic—if the API fails, the code should wait 5 seconds and try again twice before giving up.

Task 4: The "Vault" Database Migration (SQL)

- **The Task:** Migrate the Phase 1 "Login System" from a text file to a structured SQL database.
- **Deep Details:**

- **Library:** Use sqlite3.
- **Schema Design:** Create a table with specific constraints (e.g., Username must be UNIQUE and NOT NULL).
- **Security:** Use "Parameterized Queries" (e.g., cursor.execute("SELECT * FROM users WHERE id=?", (user_id,))) to prevent SQL Injection attacks.

Task 5: The "Janitor" 2.0 (Subprocess Management)

- **The Task:** Create a system maintenance tool that interacts directly with the Operating System.
- **Deep Details:**
 - **Process Control:** Use the subprocess module to run terminal commands like df -h (disk space) or ping.
 - **Data Parsing:** Capture the "Standard Output" (stdout) of the terminal command, parse the text using Python, and save a summary report as a JSON file.
 - **Error Capture:** Use stderr to catch and log system-level permission errors.

Task 6: Test-Driven Inventory Suite (Pytest)

- **The Task:** Write a full suite of automated tests for a commercial inventory system.
- **Deep Details:**
 - **Framework:** Use pytest instead of manual checking.
 - **Edge Cases:** Write tests for "Impossible" scenarios—e.g., adding a negative quantity or removing an item that doesn't exist.
 - **Fixtures:** Use @pytest.fixture to create a temporary "Mock Database" that resets after every test run.

Task 7: The Asynchronous Web Monitor

- **The Task:** Build a non-blocking script that monitors the "Up/Down" status of 10 different websites at once.
- **Deep Details:**
 - **Logic:** Use asyncio and aiohttp.

- **Event Loop:** The script must send all 10 requests at the exact same time without waiting for the first one to finish.
- **Reporting:** Print the response time in milliseconds for each site. This is a bridge to high-speed backend engineering.

Task 8: Standardized CLI "Agile" Tool

- **The Task:** Build a professional command-line tool that accepts arguments from the terminal.
- **Deep Details:**
 - **Library:** Use argparse or click.
 - **Interface:** The user should be able to run `python tool.py --sort name --dir ./data`.
 - **Help Menu:** The program must automatically generate a "Help" menu when the user types `--help`.

Task 9: Collaborative Git "Merge" Mastery

- **The Task:** Simulate a professional team environment using Git.
- **Deep Details:**
 - **Workflow:** The student must create a repository, create a "Feature Branch," and push code.
 - **Conflict Resolution:** The instructor provides a code snippet that conflicts with the student's code. The student must use the terminal to perform a git merge, manually fix the "Conflict Markers," and push the final working version.

Task 10: The "Final Boss" Micro-Service

- **The Task:** Build a "Currency Watcher" service that combines all Phase 2 skills.
- **Deep Details:**
 - **Integration:** The tool must:
 1. Fetch rates from an API (**Requests**).
 2. Run every 60 seconds without blocking the terminal (**Asyncio**).
 3. Save every change to a database (**SQLite**).

4. Log any internet failures to a file (**Logging**).
5. Be installable as a global command (**Packaging**).